

Assignment: Neo4j Mini GraphRAG Retriever (Python + Cypher + JSON + Insights)

Goal

Build a **single Python script** that:

1. takes **Patient ID and/or Disease name** as user input
2. runs **2 practical Neo4j Cypher queries**
3. outputs results as **nested JSON evidence packs**
4. additionally **flattens/explodes** the JSON into **meaningful insight tables** (pandas)

What participants will build

A script: `neo4j_patient_insights.py`

Inputs (from user)

- Patient ID (optional): e.g., P_001
- Disease (optional): e.g., Hypertension or ICD-10 I10
- Time window (optional): default 90 days

Outputs

1. **Use-case A output:** nested JSON (patient summary + evidence)
2. **Use-case B output:** nested JSON (disease cohort / trends)
3. **Insights view:** pandas DataFrames created by flattening/exploding JSON (printed to console)

Use-cases (choose exactly these two)

Use-case A — Patient “Clinical Snapshot + Evidence”

Real question:

“Show the last 5 encounters for this patient, with diagnoses, medications, and note snippets.”

Expected nested JSON sections

- patient
- encounters[] with diagnoses[], medications[], notes[]
- provenance (query names, timestamps optional)

Use-case B — Disease “Newly Diagnosed Cohort”

Real question:

“List patients newly diagnosed with this disease in the last 90 days (first-ever occurrence).”

Expected nested JSON sections

- disease (name / icd10)
- cohort[] with patient_id, name, first_dx_date
- summary stats (count)

Requirements

Functional

- Must use **neo4j Python driver**
- Must load Neo4j creds from .env
- Must support either:
 - patient-only input → run Use-case A
 - disease-only input → run Use-case B
 - both → run both

Output format

- Print nested JSON to console (pretty printed)
- Print at least **2 insight tables** (DataFrames) derived from the JSON:
 - Table 1: encounter-level rows exploded (one row per encounter)
 - Table 2: diagnosis/medication exploded (one row per dx/med, with encounter info)

JSON flattening / explosion

Must use at least:

- pd.json_normalize
- explode() (at least once)

Provided Cypher templates (participants must use/adapt)

Cypher 1: Patient Summary (last N encounters)

```
MATCH (p:Patient {patient_id:$pid})-[:HAD_ENCOUNTER]->(e:Encounter)
OPTIONAL MATCH (e)-[:HAS_DIAGNOSIS]->(d:Diagnosis)
OPTIONAL MATCH (e)-[:PRESCRIBED]->(m:Medication)
OPTIONAL MATCH (n:ClinicalNote)-[:ABOUT_ENCOUNTER]->(e)
WITH p, e,
    collect(DISTINCT {icd10:d.icd10, name:d.name}) AS diagnoses,
    collect(DISTINCT {rxnorm:m.rxnorm, name:m.name, dose:pr.dose,
frequency:pr.frequency}) AS meds,
    collect(DISTINCT {note_id:n.note_id, snippet:substring(n.text,0,160)}) AS notes
ORDER BY e.date DESC
WITH p, collect({
    enc_id:e.enc_id, date:e.date, reason:e.reason,
    bp_systolic:e.bp_systolic, bp_diastolic:e.bp_diastolic, hba1c:e.hba1c,
    diagnoses:diagnoses, medications:meds, notes:notes
})[0..$limit] AS last_encounters
RETURN {
    patient: {patient_id:p.patient_id, name:p.name, dob:p.dob, sex:p.sex},
    encounters: last_encounters
} AS patient_summary;
```

Cypher 2: Newly Diagnosed Cohort (last N days)

```
MATCH (p:Patient)-[:HAD_ENCOUNTER]->(e:Encounter)-[:HAS_DIAGNOSIS]->(dx:Diagnosis)
```

```

WHERE (dx.icd10 = $icd10 OR toLower(dx.name) CONTAINS toLower($disease_name))

AND e.date >= date() - duration({days:$days})

WITH p, min(e.date) AS first_dx_date

OPTIONAL MATCH (p)-[:HAD_ENCOUNTER]->(e2:Encounter)-[:HAS_DIAGNOSIS]-
>(dx2:Diagnosis)

WHERE (dx2.icd10 = $icd10 OR toLower(dx2.name) CONTAINS
toLower($disease_name))

AND e2.date < first_dx_date

WITH p, first_dx_date, count(e2) AS prior_count

WHERE prior_count = 0

RETURN p.patient_id AS patient_id, p.name AS name, first_dx_date AS first_dx_date

ORDER BY first_dx_date DESC;

```

Insight tables to generate (must do both)

Insight Table 1 — Encounter Overview (explode encounters)

From patient_summary.encounters[], create a DataFrame with columns like:

- enc_id, date, reason, bp_systolic, bp_diastolic, hba1c
- dx_count, med_count, note_count (derived)

Insight Table 2 — Medication & Diagnosis Breakdown (double explode)

Create two DataFrames (or one combined):

- one row per **encounter + diagnosis**
- one row per **encounter + medication**

This is how you convert nested JSON into analytical views.

Suggested script structure: implement following functions (recommended)

- load_env()
- connect_neo4j()

- `run_query(cypher, params)`
- `build_patient_evidence(pid)`
- `build_disease_cohort(disease_name/icd10, days)`
- `to_insight_tables(patient_json, cohort_rows)`
- `main()` reads input and prints outputs

Deliverables

1. `neo4j_patient_insights.py`
2. `.env.example` (no secrets)
3. Screenshot or pasted console output showing:
 - nested JSON evidence pack
 - two insight tables